JAVA PROGRAMMING

UNIT 2 – INHERITANCE, PACKAGES AND INTERFACES

Syllabus: Inheritance – basic concepts, member access and inheritance, practical example, inheritance types, super, constructors, method overriding, dynamic method dispatch, abstract classes, final with inheritance. Packages and Interfaces – packages, packages and member access, importing packages, interfaces, default interface methods, static methods in an interface, private interface methods.

INHERITANCE

1. Inheritance Basics

Inheritance is the mechanism by which one class acquires the members (fields and methods) of another class. The class that is inherited is called the **superclass** (parent / base class). The class that does the inheriting is called the **subclass** (child / derived class). A subclass is a specialised version of its superclass: it inherits all members of the superclass and adds its own unique members. It models an **IS-A** relationship (a Dog IS-A Animal).

Why inheritance?

- **Code reuse** – common behaviour is written once in the parent.
- **Modelling the real world** – Vehicle -> Car, Bike.
- **Runtime polymorphism** – code written for the parent type works with any child at run time.
- **Maintainability and extension** – new child classes can be added without changing the parent.

Syntax

```
class subclass-name extends superclass-name {
    // body of the subclass
}
```

- The keyword **extends** creates a subclass.
- A subclass can have **only one** direct superclass (Java does not support multiple inheritance of classes).
- Constructors are **not** inherited; private members are not accessible in the subclass.
- Every class without an explicit `extends` implicitly extends `java.lang.Object`, so `Object` is the root of every class hierarchy.

```
// Create a superclass
class A {
    int i, j;
    void showij() {
        System.out.println("i and j: " + i + " " + j);
    }
}

// Create a subclass by extending class A
class B extends A {
    int k;
    void showk() {
        System.out.println("k: " + k);
    }
    void sum() {
        System.out.println("i+j+k: " + (i + j + k));
    }
}

class SimpleInheritance {
    public static void main(String[] args) {
        A superOb = new A();
        B subOb = new B();

        superOb.i = 10;
        superOb.j = 20;
        System.out.println("Contents of superOb:");
        superOb.showij();

        subOb.i = 7;   // subclass can use the members of A
        subOb.j = 8;
        subOb.k = 9;
        System.out.println("Contents of subOb:");
        subOb.showij();
        subOb.showk();
        System.out.println("Sum of i, j and k in subOb:");
        subOb.sum();
    }
}
```

Output:

```
Contents of superOb:
i and j: 10 20
Contents of subOb:
i and j: 7 8
k: 9
Sum of i, j and k in subOb:
i+j+k: 24
```

The subclass `B` includes all members of `A`. That is why `subOb` can access `i`, `j` and call `showij()`; inside `sum()`, `i` and `j` are used directly as if they were declared in `B`. The superclass can still be used on its own (`superOb`).

A superclass reference can refer to a subclass object

A reference variable of the superclass type can refer to an object of any subclass derived from it (`A r = new B();`). Through that reference only the members *declared in the superclass* are accessible. This fact is the basis of dynamic method dispatch (Section 8).

2. Member Access and Inheritance

Although a subclass includes **all** members of its superclass, it **cannot access** members that are declared **private**. The private members exist inside the object, but only code in the same class can use them directly.

Modifier in superclass	Accessible in the subclass?
private	No (use public/protected getters and setters of the superclass)
default (no modifier)	Yes, only if the subclass is in the **same package**

Modifier in superclass	Accessible in the subclass?
protected	Yes, even if the subclass is in a **different package**
public	Yes, everywhere

Example: private members stay private (compile-time error)

```
class A {
    int i;            // default access
    private int j;    // private to A
    void setij(int x, int y) { i = x; j = y; }
}

class B extends A {
    int total;
    void sum() {
        total = i + j;   // ERROR: j is not accessible here
    }
}

Access.java:22: error: j has private access in A
```

Since `j` is private, it is accessible only by other members of its own class. The usual fix is to keep the data private and give the subclass a controlled way in:

```
class A {
    int i;           // default access
    private int j;   // private to A
    void setij(int x, int y) { i = x; j = y; }
    int getj() { return j; } // public door to the private data
}

class B extends A {
    int total;
    void sum() {
        // total = i + j;        // ERROR: j has private access in A
        total = i + getj();      // OK: use the accessor method
    }
}

class AccessDemo {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.setij(10, 12);
        subOb.sum();
        System.out.println("Total is " + subOb.total);
    }
}
```

Output:

```
Total is 22
```

Access across packages (protected and public)

A subclass in a different package needs **public** or **protected** to use the parent's members. (Packages are explained in Section 11.)

```
package p1;
public class Parent {
    private int a = 1;       // only Parent
    int d = 4;               // default: p1 only
    protected int b = 2;     // p1 + subclasses anywhere
    public int c = 3;        // everywhere
}

package p2;
import p1.Parent;

public class Child extends Parent {
    void show() {
        // System.out.println(a);   // ERROR: private
        // System.out.println(d);   // ERROR: default, other package
        System.out.println(b);      // OK: protected via inheritance
        System.out.println(c);      // OK: public
    }
}
```

3. A More Practical Example (Box and BoxWeight)

The `Box` class from Unit 1 (width, height, depth, `volume()`) is extended to include a fourth component, **weight**. Instead of rewriting Box, `BoxWeight` inherits everything from Box and only adds `weight`.

```
class Box {
    double width, height, depth;
    double volume() { return width * height * depth; }
}

// BoxWeight = Box + weight
class BoxWeight extends Box {
    double weight;
    BoxWeight(double w, double h, double d, double m) {
        width = w;  height = h;  depth = d;   // inherited members
        weight = m;
    }
}

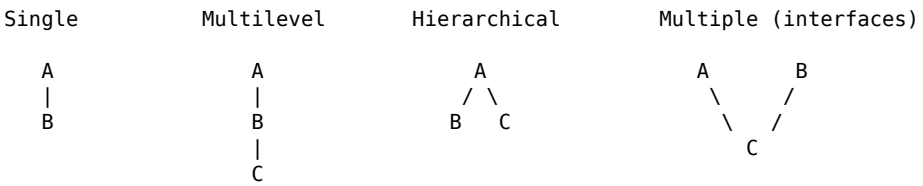
class DemoBoxWeight {
    public static void main(String[] args) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
        System.out.println("Volume of mybox1 is " + mybox1.volume());
        System.out.println("Weight of mybox1 is " + mybox1.weight);
        System.out.println("Volume of mybox2 is " + mybox2.volume());
        System.out.println("Weight of mybox2 is " + mybox2.weight);
    }
}
```

Output:

```
Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3
Volume of mybox2 is 24.0
Weight of mybox2 is 0.076
```

> **Important:** this version works only because width, height and depth are not private in Box. If Box declares them `private` (good encapsulation), the BoxWeight constructor can no longer assign them directly. The solution is `super(...)`, which is explained next.

4. Types of Inheritance



Type	Meaning	In Java
Single	A class extends one class	Supported
Multilevel	A <- B <- C: a class extends a class that itself extends another	Supported
Hierarchical	Several subclasses extend one parent	Supported
Multiple	A class has more than one parent class	NOT supported for classes (avoids the **diamond problem**); achieved using **interfaces**
Hybrid	Combination of the above	Achieved through classes + interfaces

```
class Animal {
    void eat() { System.out.println("eating..."); }
}
class Dog extends Animal {          // Single: Animal -> Dog
    void bark() { System.out.println("barking..."); }
}
class Puppy extends Dog {           // Multilevel: Animal -> Dog -> Puppy
    void weep() { System.out.println("weeping..."); }
}
class Cat extends Animal {          // Hierarchical: Dog and Cat share Animal
    void meow() { System.out.println("meowing..."); }
}

class TypesDemo {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();  p.bark();  p.weep();   // from Animal, Dog, Puppy
        Cat c = new Cat();
        c.eat();  c.meow();
    }
}
```

Output:

```
eating...
barking...
weeping...
eating...
meowing...
```

Multiple inheritance using interfaces (`class C implements A, B`) is shown in Section 14.

5. Using super

Whenever a subclass needs to refer to its **immediate superclass**, it uses the keyword **super**. It has two general forms:

- `super(arg-list);` – calls a constructor of the superclass.
- `super.member` – accesses a field or method of the superclass that has been hidden by a member of the subclass.

(a) Using super to call superclass constructors

- `super()` must always be the **first statement** inside the subclass constructor.
- Since constructors can be overloaded, `super()` can call any form of the superclass constructor; the one whose parameters match the arguments is executed.
- It always refers to the superclass **immediately above** the calling class, even in a multilevel hierarchy.
- A superclass variable can reference a subclass object, so `super(ob)` can be passed a BoxWeight object to `Box(Box ob)`.

```
class Box {
    private double width, height, depth;
```

```
    Box(Box ob) {                    // clone constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
    Box(double w, double h, double d) {
        width = w; height = h; depth = d;
    }
    Box() { width = height = depth = -1; }   // uninitialized box
    Box(double len) { width = height = depth = len; }  // cube

    double volume() { return width * height * depth; }
}

class BoxWeight extends Box {
    double weight;

    BoxWeight(BoxWeight ob) {        // clone constructor
        super(ob);
        weight = ob.weight;
    }
    BoxWeight(double w, double h, double d, double m) {
        super(w, h, d);              // call Box(w, h, d)
        weight = m;
    }
    BoxWeight() {
        super();                     // call Box()
        weight = -1;
    }
    BoxWeight(double len, double m) {
        super(len);                  // call Box(len)
        weight = m;
    }
}

class DemoSuper {
    public static void main(String[] args) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox3 = new BoxWeight();
        BoxWeight mycube = new BoxWeight(3, 2);
        BoxWeight myclone = new BoxWeight(mybox1);

        System.out.println("mybox1: " + mybox1.volume() + ", " + mybox1.weight);
        System.out.println("mybox3: " + mybox3.volume() + ", " + mybox3.weight);
        System.out.println("mycube: " + mycube.volume() + ", " + mycube.weight);
        System.out.println("myclone: " + myclone.volume() + ", " + myclone.weight);
    }
}
```

Output:

```
mybox1: 3000.0, 34.3
mybox3: -1.0, -1.0
mycube: 27.0, 2.0
myclone: 3000.0, 34.3
```

(b) Using super to access hidden members

When a subclass member has the same name as a superclass member, the subclass member **hides** the superclass one. `super.member` reaches the hidden superclass member.

```
class A {
    int i;
}

class B extends A {
    int i;                  // this i hides the i in A
    B(int a, int b) {
        super.i = a;        // i in A
        i = b;              // i in B
    }
    void show() {
        System.out.println("i in superclass: " + super.i);
        System.out.println("i in subclass: " + i);
    }
}

class UseSuper {
    public static void main(String[] args) {
        B subOb = new B(1, 2);
        subOb.show();
    }
}
```

Output:

```
i in superclass: 1
i in subclass: 2
```

`super` can also call a superclass **method** that has been overridden:

```
class Animal {
    void eat() { System.out.println("Animal is eating"); }
}

class Dog extends Animal {
    void eat() { System.out.println("Dog is eating"); }
    void work() {
        super.eat();   // Animal's eat()
        eat();         // Dog's eat()
    }
}

class Test {
    public static void main(String[] args) {
        new Dog().work();
    }
}
```

Output:

```
Animal is eating
Dog is eating
```

6. Creating a Multilevel Hierarchy and Constructor Execution

A subclass can itself be used as a superclass of another class. Given classes A, B and C where C extends B and B extends A, class C inherits all aspects of B **and** A. In the textbook, `BoxWeight` (which extends Box) is used as the superclass of `Shipment`, which adds a `cost` field.

When are constructors executed?

In a class hierarchy, constructors complete their execution **in order of derivation, from superclass to subclass**.

- Because `super()` must be the first statement of a subclass constructor, this order is the same whether or not `super()` is written explicitly.
- If `super()` is not written, Java inserts a call to the **default (no-argument)** constructor of the superclass automatically.
- If the superclass has **only parameterised** constructors, the subclass must call `super(args)` explicitly, otherwise a compile-time error occurs.

```
class A {
    A() { System.out.println("Inside A's constructor."); }
}

class B extends A {
    B() { System.out.println("Inside B's constructor."); }
}

class C extends B {
    C() { System.out.println("Inside C's constructor."); }
}

class CallingCons {
    public static void main(String[] args) {
        C c = new C();
    }
}
```

Output:

```
Inside A's constructor.
Inside B's constructor.
Inside C's constructor.
```

```
class Parent {
    Parent(int a) {                      // no no-arg constructor
        System.out.println("Parent constructor, a = " + a);
    }
}

class Child extends Parent {
    Child() {
        super(100);   // compulsory, otherwise compile-time error
        System.out.println("Child constructor");
    }
}

public class ConstructorInheritance {
    public static void main(String[] args) {
        new Child();
    }
}
```

Output:

```
Parent constructor, a = 100
Child constructor
```

7. Method Overriding

In a class hierarchy, when a method in a subclass has the **same name and type signature** as a method in its superclass, the subclass method **overrides** the superclass method. When an overridden method is called through a subclass object, the subclass version is always used; the superclass version is hidden.

Rules of overriding

- Same method name and same parameter list (same signature).
- Return type must be the same or a **covariant** (sub-)type.
- The access level must not be reduced (a public method cannot be overridden as protected or private).
- Overriding needs an inheritance (IS-A) relationship – it happens between two classes.
- **final**, **static** and **private** methods cannot be overridden (a static method with the same signature in a subclass only *hides* the parent one).
- An overriding method cannot throw new or broader **checked** exceptions than the overridden method (exceptions are covered in Unit 3).
- Use the `@Override` annotation: the compiler reports an error if the method does not actually override something.

Method overloading vs method overriding

Method Overloading	Method Overriding
Increases the readability of the program.	Provides a specific implementation of a method already provided by the superclass.
Performed **within** a class.	Occurs in **two classes** that have an IS-A (inheritance) relationship.
Parameter lists must be different.	Parameter lists must be the same.
Compile-time polymorphism.	Run-time polymorphism.
Cannot be done by changing only the return type.	Return type must be the same or covariant.

Example: real-life overriding (Bank)

```
class Bank {
    int getRateOfInterest() { return 0; }
}
class SBI extends Bank {
    @Override
    int getRateOfInterest() { return 8; }
}
class ICICI extends Bank {
    @Override
    int getRateOfInterest() { return 7; }
}
class AXIS extends Bank {
    @Override
    int getRateOfInterest() { return 9; }
}

class Test2 {
    public static void main(String[] args) {
        SBI s = new SBI();
        ICICI i = new ICICI();
        AXIS a = new AXIS();
        System.out.println("SBI Rate of Interest: " + s.getRateOfInterest());
        System.out.println("ICICI Rate of Interest: " + i.getRateOfInterest());
        System.out.println("AXIS Rate of Interest: " + a.getRateOfInterest());
    }
}
```

Output:

```
SBI Rate of Interest: 8
ICICI Rate of Interest: 7
AXIS Rate of Interest: 9
```

Calling the superclass version with super

```
class A {
    void show() { System.out.println("A.show()"); }
}

class B extends A {
    @Override
    void show() {
        super.show();                    // run the superclass version first
        System.out.println("B.show()");  // then extend it
    }
}

class OverrideSuper {
    public static void main(String[] args) {
        new B().show();
    }
}
```

Output:

```
A.show()
B.show()
```

Why overridden methods? Overriding lets a general class define the methods common to all its subclasses while each subclass supplies its own specific implementation. This is Java's way of supporting "one interface, multiple methods" (run-time polymorphism).

8. Dynamic Method Dispatch

Dynamic method dispatch is the mechanism by which a call to an **overridden method is resolved at run time**, rather than at compile time. It is how Java implements **run-time polymorphism**.

- A superclass reference variable can refer to a subclass object.
- When an overridden method is called through that reference, Java decides **which version to run based on the type of the object being referred to** at the time of the call, **not** the type of the reference variable.
- Only **instance** methods that are overridden take part in dispatch. Static methods, private methods and final methods are bound at compile time.
- **Fields are not polymorphic**: a field access is resolved using the reference type.

```
class A {
    void callme() { System.out.println("Inside A's callme method"); }
}
class B extends A {
    @Override
    void callme() { System.out.println("Inside B's callme method"); }
}
class C extends A {
    @Override
    void callme() { System.out.println("Inside C's callme method"); }
}

class Dispatch {
    public static void main(String[] args) {
        A a = new A();   // object of type A
        B b = new B();   // object of type B
        C c = new C();   // object of type C
        A r;             // reference of the superclass type

        r = a;  r.callme();   // calls A's version
        r = b;  r.callme();   // calls B's version
        r = c;  r.callme();   // calls C's version
    }
}
```

Output:

```
Inside A's callme method
Inside B's callme method
Inside C's callme method
```

Had the version been chosen by the type of the reference variable `r`, all three calls would have printed A's message. The output shows that the object type decides.

Methods are dynamic, fields are not

```
class P {
    int x = 10;
    void show() { System.out.println("P.show()"); }
}
class Q extends P {
    int x = 20;
    @Override
    void show() { System.out.println("Q.show()"); }
}

class FieldDemo {
    public static void main(String[] args) {
        P p = new Q();               // P reference, Q object
        p.show();                    // method: decided by the object -> Q
        System.out.println("x = " + p.x);  // field: decided by reference -> P
    }
}
```

Output:

```
Q.show()
x = 10
```

9. Using Abstract Classes

Sometimes a superclass defines only the general form of an operation and leaves the details to its subclasses. Such a method is declared **abstract** and has no body.

```
abstract type name(parameter-list);        // no method body

abstract class ClassName {                  // class containing abstract methods
    ...
}
```

Rules

- Any class that contains one or more abstract methods must itself be declared `abstract`.
- An abstract class **cannot be instantiated** with `new`; an abstract class reference can however refer to a subclass object.
- It can have fields, constructors, concrete (implemented) methods and abstract methods.
- You cannot declare abstract constructors or abstract `static` methods.
- A subclass must either implement **all** abstract methods of the superclass or be declared abstract itself.
- An abstract class cannot be `final` (abstract needs subclasses, final forbids them).

```
abstract class Figure {
    double dim1, dim2;
    Figure(double a, double b) { dim1 = a; dim2 = b; }
    abstract double area();          // no body
}

class Rectangle extends Figure {
    Rectangle(double a, double b) { super(a, b); }
    @Override
    double area() {
        System.out.println("Inside Area for Rectangle.");
        return dim1 * dim2;
    }
}

class Triangle extends Figure {
    Triangle(double a, double b) { super(a, b); }
    @Override
    double area() {
        System.out.println("Inside Area for Triangle.");
        return dim1 * dim2 / 2;
    }
}

class AbstractAreas {
    public static void main(String[] args) {
        // Figure f = new Figure(10, 10);   // illegal: abstract class
        Rectangle r = new Rectangle(9, 5);
        Triangle t = new Triangle(10, 8);
        Figure figref;                      // reference is allowed
        figref = r;
        System.out.println("Area is " + figref.area());
        figref = t;
        System.out.println("Area is " + figref.area());
    }
}
```

Output:

```
Inside Area for Rectangle.
Area is 45.0
Inside Area for Triangle.
Area is 40.0
```

10. Using final with Inheritance

The keyword `final` has three uses. The first creates a named constant; the other two apply to inheritance.

Use	Effect	Example
final variable	Value cannot be changed after it is assigned (constant)	`final int MAX = 100;`
final method	Cannot be **overridden** by a subclass	`final void meth() { }`
final class	Cannot be **inherited** (no subclass). All its methods are implicitly final	`final class A { }`

```
class A {
    final void meth() { System.out.println("This is a final method."); }
}
class B extends A {
    void meth() { }          // ERROR! Can't override.
}

final class A { }
class B extends A { }            // ERROR! Can't subclass A
```

```
class Parent {
    final void fixed() { System.out.println("fixed - cannot be overridden"); }
}

final class Utility {                 // cannot be extended
    static void print() { System.out.println("Utility method"); }
}

class FinalDemo {
    public static void main(String[] args) {
        final int MAX_MARKS = 100;     // constant
        System.out.println(MAX_MARKS);
        Utility.print();
        new Parent().fixed();
    }
}
```

Output:

```
100
Utility method
fixed - cannot be overridden
```

- Use `final` methods to prevent the behaviour from being changed, and final classes to prevent extension (for example `java.lang.String` is final).
- It is illegal to declare a class both `abstract` and `final`.
- Calls to final methods can be resolved at compile time (early binding), which can give a small performance benefit.

PACKAGES AND INTERFACES

11. Packages

In the programs so far, every class name came from the same name space, so each class needed a unique name. Without some way to manage the name space you would run out of convenient names, and your class names could collide with names chosen by other programmers. **Packages** solve this: a package is a container that groups related classes and interfaces and defines a name space for them.

Defining a package

- Include a `package` statement as the **first statement** in the source file. Every class in that file belongs to the package.
- If the package statement is omitted, the classes go into the **default package** (which has no name).
- Java uses **file-system directories** to store packages: the .class files of package `mypack` must be in a directory called `mypack`. Case is significant and the directory name must match the package name exactly.
- A package hierarchy is created with periods and must be mirrored in the directories.

```
package pkg;                    // general form
package mypack;                 // example
package pkg1[.pkg2[.pkg3]];     // multilevel
package java.awt.image;         // stored in java\awt\image
```

Finding packages and CLASSPATH

How does the run-time system know where to find your packages? There are three ways:

- By default, it starts from the **current working directory** – a package in a subdirectory of the current directory is found.
- Set the **CLASSPATH** environment variable to the directory (or directories) that contain the package.
- Use the `-classpath` option with `java` and `javac`.

> The class path must point to the directory **above** the package, not to the package itself. If the package is `C:\MyPrograms\Java\mypack`, the class path is `C:\MyPrograms\Java`.

Example: creating and using a package

```
// File: mypack/Balance.java
package mypack;

public class Balance {
    String name;
    double bal;

    public Balance(String n, double b) {
        name = n;
        bal = b;
    }
    public void show() {
        if (bal < 0) System.out.print("--> ");
        System.out.println(name + ": $" + bal);
    }
}

// File: TestBalance.java  (in the directory above mypack)
import mypack.Balance;          // or: import mypack.*;

class TestBalance {
    public static void main(String[] args) {
        Balance test = new Balance("J. J. Jaspers", 99.88);
        test.show();
    }
}
```

```
Desktop\
  TestBalance.java
  mypack\
    Balance.java

C:\Users\...\Desktop> javac mypack/Balance.java
C:\Users\...\Desktop> javac TestBalance.java
C:\Users\...\Desktop> java TestBalance

J. J. Jaspers: $99.88
```

A class and its members must be **public** to be used from another package. If a class that is part of a package has a main() method, it must be run with its package name: `java mypack.AccountBalance` from the directory above `mypack` – plain `java AccountBalance` will not work.

Benefits of packages

- Prevents name-space collision (same class name in different packages).
- Groups related classes and interfaces into a single unit, making it easy to find a class.
- Gives greater control over the source code and helps to separate your work from library code.
- Provides access protection (Section 12).

Types of packages

- **Predefined (built-in) packages** – standard packages that come with the JRE.
- **User-defined packages** – created by the programmer to bundle related classes.

Package	Contents
java.lang	Core language classes: Object, String, System, Math, wrapper classes, threads, exceptions. **Imported automatically**.
java.util	Utility classes: Date, Scanner, collection classes (ArrayList, HashMap ...)
java.io	Input and output classes for streams and files
java.net	Networking support: Socket, URL
java.awt	Classes for GUI: windows, buttons, menus
java.applet	Applet class and related interfaces

Static import

Static import imports the *static members* of a class, so the class name and dot are not needed when using them. For example, `sqrt()` can be used instead of `Math.sqrt()`.

```
import static java.lang.Math.*;
import static java.lang.System.*;

class StaticImportTest {
    public static void main(String[] args) {
        out.println(sqrt(4));
        out.println(pow(2, 2));
        out.println(abs(6.3));
    }
}
```

Output:

```
2.0
4.0
6.3
```

12. Packages and Member Access

Classes and packages are both means of encapsulating and containing the name space and scope of variables and methods. Packages act as containers for classes and other packages; classes act as containers for data and code. Because of this interplay, Java provides four categories of visibility for class

members: subclasses in the same package, non-subclasses in the same package, subclasses in different packages, and classes that are neither in the same package nor subclasses.

	private	No modifier	protected	public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

- Anything declared **public** can be accessed from anywhere.
- Anything declared **private** cannot be seen outside its class.
- With **no modifier** (default access), a member is visible to its own package, including subclasses there.
- **protected** lets an element be seen outside the package, but only by subclasses.

Access levels of a class

- A top-level (non-nested) class has only two possible access levels: **default** and **public**.
- A public class is accessible by any other code; a class with default access is accessible only within its own package.
- A source file can contain only **one public class**, and the file name must be the same as that class name.

13. Importing Packages

To use a class that is in another package you either use its fully qualified name or an **import** statement. Once imported, the class can be referred to directly by its simple name.

```
import pkg1[.pkg2].(classname | *);

import java.util.Date;      // one class
import java.io.*;           // all classes of the package
```

- **Position:** import statements come immediately after the package statement (if any) and before any class definition.
- There is no practical limit on the depth of a package hierarchy, except that imposed by the file system.
- `*` imports all classes of the package (not the sub-packages).
- `java.lang` is **implicitly imported** into every program.
- The import statement is **optional**: wherever a class name is used, its fully qualified name can be used instead, e.g. `class MyDate extends java.util.Date`.

Name-space collision

If two imported packages contain a class with the same name, use the fully qualified name to avoid ambiguity:

```
package pack1;               package pack2;
public class Student { }     public class Student { }

// In another file:
import pack1.*;
import pack2.*;

class Demo {
    pack1.Student s1;        // fully qualified names remove the ambiguity
    pack2.Student s2;
}

package order:    package statement  ->  import statements  ->  class / interface declarations
```

14. Interfaces

Using the keyword **interface**, you can fully abstract a class's interface from its implementation: you specify **what** a class must do, but not **how** it does it. An interface is syntactically similar to a class, but it has no instance variables and its (traditional) methods are declared without a body. Once defined, any number of classes can implement an interface, and one class can implement any number of interfaces.

14.1 Defining an interface

```
access interface name {
    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);
    type final-varname1 = value;
    type final-varname2 = value;
}

interface Callback {
    void callback(int param);
}
```

- **access** is either `public` or default (package-level).
- Methods are implicitly `public abstract`.
- Variables are implicitly `public static final` (constants) and must be initialised.
- An interface cannot be instantiated and cannot have instance variables or constructors.

14.2 Implementing interfaces

```
class classname [extends superclass] [implements interface [, interface...]] {
    // class-body
}
```

- A class uses the `implements` clause and must provide all methods of the interface. If it implements more than one interface, they are separated by commas.
- Methods that implement an interface **must be declared public**, and their signature must match exactly the one in the interface.
- If a class does not implement all methods, it must be declared `abstract`.
- A class can define additional members of its own.
- If a class implements two interfaces that declare the same method, the same implementation serves both.

14.3 Accessing implementations through interface references

You can declare a variable of an *interface type*. Any object of any class that implements the interface can be assigned to it. The method called is looked up **dynamically at run time**, so the calling code works without knowing anything about the actual class. An interface reference knows only the methods declared by its interface, so `c.nonIfaceMeth()` is not allowed.

```
interface Callback {
    void callback(int param);
}

class Client implements Callback {
    public void callback(int p) {        // must be public
        System.out.println("callback called with " + p);
    }
    void nonIfaceMeth() {                // extra member of the class
        System.out.println("Classes that implement interfaces "
                           + "may also define other members, too.");
    }
}

class AnotherClient implements Callback {
    public void callback(int p) {
        System.out.println("Another version of callback");
        System.out.println("p squared is " + (p * p));
    }
}

class TestIface {
    public static void main(String[] args) {
        Callback c = new Client();       // interface reference
        c.callback(42);
        // c.nonIfaceMeth();             // ERROR: not part of Callback

        c = new AnotherClient();         // same reference, new object
        c.callback(42);
    }
}
```

Output:

```
callback called with 42
Another version of callback
p squared is 1764
```

14.4 Multiple inheritance using interfaces

A class can implement several interfaces, which gives Java the benefit of multiple inheritance without the diamond problem, because interfaces (traditionally) carry no implementation state.

```
interface Printable { void print(); }
interface Showable  { void show(); }

class Document implements Printable, Showable {
    public void print() { System.out.println("Printing document"); }
    public void show()  { System.out.println("Showing document"); }
}

class MultipleDemo {
    public static void main(String[] args) {
        Document d = new Document();
        d.print();
        d.show();
    }
}
```

Output:

```
Printing document
Showing document
```

14.5 Nested interfaces

An interface can be declared as a member of a class or of another interface; this is a **member (nested) interface**. A nested interface can be `public`, `private` or `protected` (a top-level interface can only be public or default). Outside its enclosing scope it must be qualified by the name of the enclosing class or interface.

```
class A {
    // nested (member) interface
    public interface NestedIF {
        boolean isNotNegative(int x);
    }
}

class B implements A.NestedIF {          // qualified by the enclosing class
    public boolean isNotNegative(int x) {
        return x < 0 ? false : true;
    }
}

class NestedIFDemo {
    public static void main(String[] args) {
        A.NestedIF nif = new B();
        if (nif.isNotNegative(10))
            System.out.println("10 is not negative");
        if (nif.isNotNegative(-12))
            System.out.println("this won't be displayed");
    }
}
```

Output:

```
10 is not negative
```

14.6 Variables in interfaces

An interface can be used to import **shared constants** into multiple classes. When a class implements such an interface, all the constant names are in scope as if they were imported as `final` variables. (The textbook notes this technique is controversial; it is described for completeness.)

```
interface SharedConstants {
    int NO = 0;          // public static final by default
    int YES = 1;
    int MAYBE = 2;
}

class ConstDemo implements SharedConstants {
    static void answer(int result) {
        switch (result) {
            case NO:    System.out.println("No");    break;
            case YES:   System.out.println("Yes");   break;
            case MAYBE: System.out.println("Maybe"); break;
        }
    }
    public static void main(String[] args) {
        answer(YES);
        answer(NO);
    }
}
```

Output:

```
Yes
No
```

14.7 Interfaces can be extended

One interface can inherit another using `extends` (an interface can extend several interfaces, separated by commas). A class that implements the sub-interface must implement every method of the whole inheritance chain.

```
interface A {
    void meth1();
}
interface B extends A {          // B inherits meth1() from A
    void meth2();
}

class MyClass implements B {     // must implement meth1() and meth2()
    public void meth1() { System.out.println("Implement meth1()."); }
    public void meth2() { System.out.println("Implement meth2()."); }
}

class IFExtend {
    public static void main(String[] args) {
        MyClass ob = new MyClass();
        ob.meth1();
        ob.meth2();
    }
}
```

Output:

```
Implement meth1().
Implement meth2().
```

14.8 Partial implementations

If a class includes an interface but does not fully implement its methods, the class must be declared `abstract`. A concrete subclass then completes the implementation.

```
interface Callback {
    void callback(int p);
}

abstract class Incomplete implements Callback {   // partial implementation
    void show() { System.out.println("Incomplete.show()"); }
}

class Complete extends Incomplete {
    public void callback(int p) {                 // finishes the job
        System.out.println("callback " + p);
    }
}

class PartialDemo {
    public static void main(String[] args) {
        Complete c = new Complete();
        c.show();
        c.callback(5);
    }
}
```

Output:

```
Incomplete.show()
callback 5
```

14.9 Applying interfaces

Interfaces let you separate the *specification* from the *implementation*. For example, an `IntStack` interface declares `push()` and `pop()`; a fixed-size stack and a growable stack can both implement it, and the code that uses an `IntStack` reference works with either. A complete runnable IntStack program is in Section 17.

15. Default Interface Methods

Before JDK 8, every interface method was abstract. JDK 8 added the **default method**: an interface method that provides a body, written with the keyword `default`. During development it was also called an *extension method*.

Why were default methods added?

- To **evolve interfaces without breaking existing code**: adding a new abstract method to a widely used interface would break every implementing class; a default method supplies an implementation that is used if a class does not provide its own.
- To make some methods **optional** (e.g. an optional `remove()` for read-only collections), so a class need not write a placeholder implementation.

```
interface MyIF {
    int getNumber();                        // abstract (no body)
    default String getString() {            // default method (JDK 8)
        return "Default String";
    }
}

class MyIFImp implements MyIF {             // does not override getString()
    public int getNumber() { return 100; }
}

class MyIFImp2 implements MyIF {            // overrides getString()
    public int getNumber() { return 100; }
    public String getString() { return "This is a different string."; }
}

class DefaultDemo {
    public static void main(String[] args) {
        MyIFImp o1 = new MyIFImp();
        MyIFImp2 o2 = new MyIFImp2();
        System.out.println(o1.getNumber());
        System.out.println(o1.getString());   // default version
        System.out.println(o2.getString());   // class version wins
    }
}
```

Output:

```
100
Default String
This is a different string.
```

- A default method may be overridden by an implementing class, but it is not required.
- Default methods do **not** change a key difference: an interface still cannot have instance variables (it cannot maintain state), and it still cannot be instantiated by itself.
- Default methods give a limited form of multiple inheritance of *behaviour*, but not of state.

Multiple inheritance issues: conflict resolution rules

If a class implements two interfaces that both provide a default method with the same name, Java uses these rules:

- **Rule 1:** A **class implementation always wins** over an interface default method.
- **Rule 2:** If a class implements two interfaces with the same default method and does **not** override it, the result is a **compile-time error**.
- **Rule 3:** If one interface extends another and both define the same default method, the **more specific (inheriting) interface's** version is used.
- A specific interface's default can be called explicitly with `InterfaceName.super.methodName()`.

```
interface Alpha {
    default void reset() { System.out.println("Alpha reset"); }
}
interface Beta {
    default void reset() { System.out.println("Beta reset"); }
}

// Without its own reset(), this class would NOT compile.
class MyClass implements Alpha, Beta {
    public void reset() {
        Alpha.super.reset();      // InterfaceName.super.method()
        Beta.super.reset();
    }
}

class ConflictDemo {
    public static void main(String[] args) {
        new MyClass().reset();
    }
}
```

Output:

```
Alpha reset
Beta reset
```

16. Use of static Methods in an Interface

JDK 8 also allows an interface to define **static** methods. Like static methods of a class, they can be called independently of any object: no implementation of the interface and no instance is needed. They are called with the interface name: `InterfaceName.staticMethodName()`.

```
interface MyIF {
    int getNumber();
    static int getDefaultNumber() {        // static method (JDK 8)
        return 0;
    }
}

class StaticIFDemo {
    public static void main(String[] args) {
        // called with the interface name - no object needed
        int defNum = MyIF.getDefaultNumber();
        System.out.println("Default number = " + defNum);
    }
}
```

Output:

```
Default number = 0
```

- Static interface methods are **not inherited** by an implementing class or by a sub-interface, so they must always be called through the interface name.

17. Private Interface Methods

Beginning with **JDK 9**, an interface can include **private** methods.

- A private interface method can be called only by a **default method** or another private method (including private static) of the **same interface**.
- It cannot be used by code outside the interface, and it is **not inherited** by sub-interfaces.
- Key benefit: two or more default methods can share common code, **avoiding code duplication**.

```
import java.util.Arrays;

interface IntStack {
    void push(int item);
    int pop();

    default int[] popNElements(int n) {
        return getElements(n);
    }
    default int[] skipAndPopNElements(int skip, int n) {
        getElements(skip);                // throw away 'skip' elements
        return getElements(n);
    }
    private int[] getElements(int n) {     // private method (JDK 9)
        int[] elements = new int[n];
        for (int i = 0; i < n; i++) elements[i] = pop();
        return elements;
    }
}

class FixedStack implements IntStack {
    private int[] stck = new int[10];
    private int tos = -1;
    public void push(int item) {
        if (tos == 9) System.out.println("Stack is full.");
        else stck[++tos] = item;
    }
    public int pop() {
        if (tos < 0) { System.out.println("Stack underflow."); return 0; }
        return stck[tos--];
    }
}

class PrivateDemo {
    public static void main(String[] args) {
        FixedStack s = new FixedStack();
        for (int i = 1; i <= 10; i++) s.push(i);
        System.out.println(Arrays.toString(s.popNElements(3)));
        System.out.println(Arrays.toString(s.skipAndPopNElements(2, 3)));
    }
}
```

Output:

```
[10, 9, 8]
[5, 4, 3]
```

Both `popNElements()` and `skipAndPopNElements()` use the private `getElements()`. Because it calls `pop()`, it automatically uses whatever `pop()` the implementing class provides, so it works for any stack class that implements IntStack.